קראו בזהירות את ההנחיות!

משך הבחינה:
3 שעות, לא כולל תוספת זמן עבור הסטודנטים הזכאים לכך.

מבנה הבחינה:
- **חלק א'** - **4 שאלות** (10 נק' כל אחת, כל שאלה מורכבת משני סעיפים). **יש לענות על כל השאלות.**
- **חלק ב'** - **6 שאלות תכנות** (15 נק' כל אחת). **יש לענות רק על 4 מתוך 6 השאלות**. בסוף המבחן תתבקשו לסמן אילו שאלות בחרתם.
- **סימון השאלות אשר בחרתם בבחינה** - **נקודה אחת** בונוס תינתן על סימון השאלות אשר בחרתם בחלק ב', בשאלה נפרדת, בסוף הבחינה. **שימו לב -** לא יהיה ניתן לקבל נקודה זאת בדיעבד, מרגע שהמבחן נסגר, בכל דרך שהיא וגם לא במייל. הקפידו למלא זאת כמה דקות לפני סיום הבחינה.

הנחיות נוספות:
- בשאלות התכנות עליכם לכתוב קוד **יעיל** (מבחינת זמן ריצה וזיכרון) ונקי ככל האפשר. תוכניות לא יעילות או שאינן כתובות היטב לא יקבלו את מלוא הנקודות גם אם הן עובדות נכון.
- בכל שאלות התכנות מותר לכתוב פונקציות עזר נוספות ללא הגבלה. **אך אין לשנות את חתימת הפונקציות (!)**
- אין צורך לתעד את הקוד. אין צורך ב-docstring. הערות לקוד במקומות בהם כתבתם משהו מורכב עשויות לסייע בבדיקת התשובה שלכם אך אינן הכרחיות.
- במידה ותענו על יותר מ-4 שאלות בחלק ב', ייבדקו 4 השאלות הראשונות.
- **הבחינה היא עם חומר סגור. אין לעשות שימוש בפלאפון מכל סוג שהוא, במחשבון, בדפי נוסחאות, או בכל חומר עזר אחר.**
- מותר לעשות שימוש במחברות טיוטא אשר יוחלקו בכיתות. אין צורך להגיש אותם בסוף הבחינה.
- בשאלות השלמת קוד בחלק א', **אין קשר בין אורך המקטע שמסמן קוד חסר, לבין כמות הקוד שצריך למלא**.
- בשאלות הקוד בחלק ב', היכן שרשום שלא ניתן לערוך את הקוד מעל השורה הנוכחית, **לא ניתן לערוך את הקוד מעל השורה הנוכחית.**
- אנא המנעו משאלות לסגל הקורס, אשר אינן קשורות ישירות להבנת השאלה. צוות הקורס לא יוכל לספק רמזים או לענות על שאלות אחרות במהלך המבחן.
- השימוש בקבצי הקול שליד חלקי הטקסט הם לזכאים להקראה **בלבד**.

בהצלחה!

שאלה **1**

לא נענה

ניקוד השאלה: 10.00

נתונות שתי הפונקציות **add1** ו-**twice**:

```
def add1(x):
    return x+1

def twice(x):
    return 2*x
```

הפונקציה הרקורסיבית **waze** מקבלת שני שלמים חיוביים **x** ו-**y**, ומחזירה את מספר האופנים (המסלולים) בהם ניתן להגיע מ-**x** ל-**y** על ידי שימוש בפונקציות **add1** ו-**twice** בלבד.

אם **x** שוה ל-**y** אין צורך לזמן את הפונקציות **add1** ו-**twice**, ובמקרה כזה **waze** תחזיר **1**.
אם **x** גדול מ-**y**, הפונקציה תחזיר **0**.

דוגמאות:

```
>>> waze(5, 5) # No calls of add(), twice() needed
1

>>> waze(12, 5) # 12>5
0

>>> waze(4,8) # twice(4), add1(add1(add1(add1(4)))
2

>>> waze(3, 5) # add1(add1(3))
1
```

---

6) A. נק') השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def waze(x, y):
    if x > y:
        return ______(1)______
    elif ______(2)______:
        return 1
    else:
        return ______(3)______ + ______(4)______
```

(1)

(2)

(3)

(4)

---

4) B. נק') בחרו לכל אחד מהביטויים הבאים את האופרטור המתאים כך שכל אחד מהביטויים יהיה נכון לכל `x, y, z` שלמים וחיוביים.

(1) waze(min(x, y), z) [ ] **waze(max(x, y), z)**

(2) waze(x, z) [ ] **waze(x, y)*waze(y, z)**

שאלה **2**

לא נענה

ניקוד השאלה: 10.00

ניתן להניח להניח ש **del_loc** גדול מאפס

להלן הגדרות המחלקות **Node** ו-**LinkedList**:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
        self.len = 0
```

השלימו את הקוד בשתי השיטות הבאות של המחלקה **LinkedList** במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד).

---

5) A. נק') השלימו את הקוד בשיטה **delete_at_loc(self, del_loc)** אשר מורידה מהרשימה את האיבר במקום ה-**del_loc**. ניתן להניח ש **del_loc** גדול מאפס.

```
    def delete_at_loc(self, del_loc):
        cur = self.head
        for i in range(______(1)______):
            cur = ______(2)______
        ______(3)______ = cur.next.next
        self.len -= 1
```

(1)

(2)

(3)

---

5) B. נק') השלימו את הקוד בשיטה **find_node_loc(self, data)** אשר מחזירה קישור (מצביע) לאיבר הראשון ברשימה המכיל את ה-**data** אשר התקבל כארגומנט, וכן את המיקום שלו ברשימה, מספר שלם, אם הוא נמצא ברשימה, ואת הערך **None** אם האיבר איננו ברשימה (האינדקס הראשון ברשימה הוא 0).

```
def find_node_loc(self, data):
    cur = self.head
    loc = 0
    while ______(1)______:
        if ______(2)______:
            return cur, loc
        else:
            cur = ______(3)______
            loc = ______(4)______
    return None
```

(1)

(2)

(3)

(4)

שאלה **3**

לא נענה

ניקוד השאלה: 10.00

5) A. נק' סמנו את כל הביטויים הנכונים (**g** מוגדרת להיות פונקציה כלשהי של **n**).

- $O(g(2n))=O(g(n)+2/n)$
- $O(n/5)=O(5n)$
- $O(2^n)=O(2^n * n^2)$
- $O(\log(2^n))=O(\log(2^n * n^2))$

---

5) B. נק' בחרו את סיבוכיות זמן הריצה של הפונקציה **f4** כפונקציה של אורך הרשימה **lst**.

```
def f4(lst):
    x, n = 0, len(lst)
    for t in range(n):
        k = 1
        while k < t:
            x += lst[k]
            k *= 3
```

שאלה **4**

לא נענה

ניקוד השאלה: 10.00

5) נק' A. (`go_flat(lst)` היא פונקציה **לא רקורסיבית** אשר "משטחת" רשימות (הופכת אותן לרשימה חד-מימדית).

הניחו שמלבד מספרים, מבני הנתונים היחידים שיכולים להימצא ב-`lst` הם רשימות. סדר האיברים ברשימת התוצאה הוא אותו סדר כמו ברשימת הקלט.

דוגמאות:

```
>>> go_flat([1,[2,[3,4],5],[6,[7,8],9],11])
[1, 2, 3, 4, 5, 6, 7, 8, 9, 11]

>>> go_flat([3, 2, 3])
[3, 2, 3]

>>> go_flat([])
[]
```

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def go_flat(lst):
    res = []
    templist = [lst]
    while templist:
        cur = templist.pop()
        if ______(1)______:
            templist.______(2)______
        else:
            res.______(3)______
    return res
```

(1)

(2)

(3)

---

5) נק' B. (`go_flat_gen(lst)` הוא **גנרטור רקורסיבי** אשר מחזיר לפי סדר את אותם האיברים ש **`go_flat(lst)`** מחזירה.

השלימו את הקוד במקומות המסומנים (תיבת מילוי בהתאם לאינדקס המסומן בקוד):

```
def go_flat_gen(lst):
    for k in lst:
        if isinstance(k, list):
            ______(1)______
        else:
            yield ______(2)______
```

(1)

(2)

שאלה **5**

לא נענה

ניקוד השאלה: 15.00

עץ מונוטוני

נתונה המחלקה **Tree**:

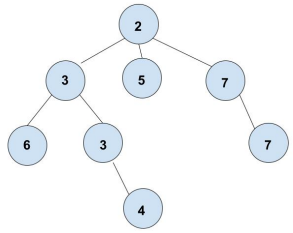
```
class Tree:
    def __init__(self, data, branches=[]):
        self.data = data
        self.branches = list(branches)

    def is_a_leaf(self):
        return not self.branches
```

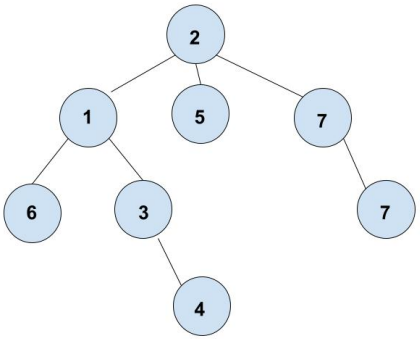
כתבו פונקציה **רקורסיבית** **monotonic(t)** אשר מקבלת עץ **t**, שערכי קודקודיו הינם מספרים, ומחזירה **True** אם ערכו של כל קדקוד **k** בעץ, קטן או שווה לערכו של כל קודקוד שבתתי העצים של **k**. אחרת הפונקציה תחזיר **False**.

לדוגמא:

עץ מונוטוני



עץ לא מונוטוני



הפונקציה תחזיר **True** עבור העץ הימני, ו-**False** עבור העץ השמאלי.

Answer:

(penalty regime: 0 %)

Reset answer

```
                                                                    :class Tree
                                                  :([]=def __init__(self, data, branches
                                                                  self.data = data
                                                      (self.branches = list(branches

                                                                          :(def is_a_leaf(self
                                                               return not self.branches

######### !DO NOT EDIT ABOVE THIS LINE #########

                                                                    :(def monotonic(t
                                                                             ...
```

שאלה **6**

לא נענה

ניקוד השאלה: 15.00

סוגריים מאוזנות

נתונים שלושה סוגי סוגריים: **()**,**[]**,**{}**.

מחרוזת שיש בה סוגריים הינה מאוזנת אם:

1. לכל סוגר שמאלי מתאים סוגר ימני מאותו סוג אשר נמצא בתוך אותה מחרוזת, במרחק מה מצד ימין שלו.

2. תת המחרוזת שבין שני הסוגריים הללו היא גם מאוזנת.

כתבו פונקציה **רקורסיבית** **balanced(str1)** אשר מקבלת מחרוזת **str1**, ומחזירה **True** אם המחרוזת מאוזנת, ו-**False** אחרת.

דוגמאות לסוגי מחרוזות מאוזנות (לצורך הבהירות אין בדוגמאות הבאות תווים אחרים מלבד הסוגריים):

```
'', '{}', '[]', '()', '{()}', '[]{}', '{[]()[(){}]}', ...
```

דוגמאות לסוגי מחרוזות שאינן מאוזנות:

```
'[', '}', '{[}', '{][}', '([)]', '((({}[]))', '(({}(()))))', ...
```

דוגמאות הרצה:

```
>>> zt = 'lu{v yf;[oh(uc)bk]bd[qn(kf);cb]j9}ry'
>>> balanced(zt)
True

>>> zt1 = 'lu{v yf;[oh(uc[bk]bd[qn(kf);cb]j9}ry'
>>> balanced(zt1)
False
```

Answer:

(penalty regime: 0 %)

Reset answer

```
def balanced(str1):
    ...
```

שאלה **7**

לא נענה

ניקוד השאלה: 15.00

עץ ביטוי

עץ ביטוי הוא עץ, לא בהכרח בינארי, שבו כל צומת פנימית מתאימה לאופרטור וכל צומת שהיא עלה מתאימה לאופרנד (הנתון עליו פועל האופרטור).

נתונה המחלקה **Tree**:

```
class Tree:
    def __init__(self, data, branches=[]):
        self.data = data
        self.branches = list(branches)

    def is_a_leaf(self):
        return not self.branches
```

כתבו פונקציה **plus_it(t)** אשר מקבלת עץ ביטוי **t**, בו האופרטורים הם רק חיבור וכפל (**'+','*'**) והאופרנדים הם מספרים שלמים חיוביים. הפונקציה תחשב את ערכו של הביטוי על ידי שימוש בפעולת חיבור בלבד, מבלי להשתמש בפעולת הכפל. כל פעולת כפל תחושב באמצעות חיבור. למשל: הפעולה **2*3** תחושב כ- **2+2+2** או **3+3**.

```
>>> t1 = Tree('*', [Tree(2), Tree(3)])
>>> print(plus_it(t1))
6

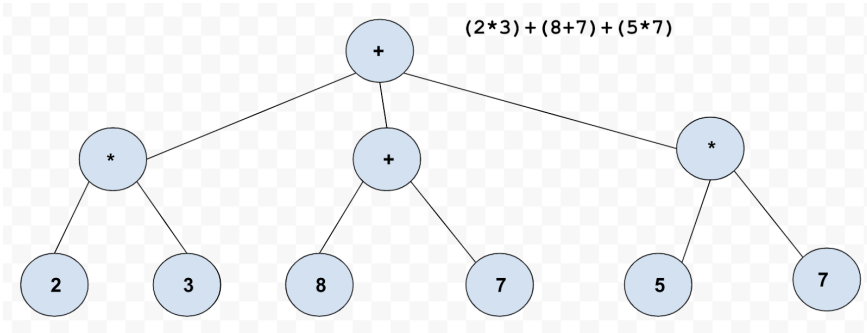
>>> t2 = Tree('+',[Tree(8), Tree(7)])
>>> print(plus_it(t2))
15

>>> t3 = Tree('*',[Tree(5), Tree(7)])
>>> print(plus_it(t3))
35

>>> t4 = Tree('+',[t1, t2, t3])
>>> print(plus_it(t4))
56

>>> t5 = Tree('*', [Tree(11), Tree(12), t4, Tree(13)])
>>> print(plus_it(t5))
96096
```

תמונה להמחשה של **t4**:



Answer:

(penalty regime: 10, 20, ... %)

Reset answer

```
                                                                    :class Tree
                                                        :([]=def __init__(self, data, branches
                                                                        self.data = data
                                                            (self.branches = list(branches

                                                                        :(def is_a_leaf(self
                                                                return not self.branches

                                                ######### !DO NOT EDIT ABOVE THIS LINE #########

                                                                                :(def plus_it(t
                                                                                        ...
```

שאלה **8**

לא נענה

ניקוד השאלה: 15.00

ייצוג בבסיס

נתונה המחלקה **Link**:

```
class Link:
    empty = ()
    def __init__(self, data, next=empty):
        self.data = data
        self.next = next

    def __repr__(self):
        if self.next is Link.empty:
            return "Link({})".format(self.data)
        else:
            return "Link({}, {})".format(self.data, self.next)
```

כידוע ניתן להציג כל מספר טבעי כסכום של חזקות שונות של 2. למשל:

```
41=32+8+1, 7=4+2+1, 21=16+4+1, ...
```

באותו אופן ניתן להציג כל מספר טבעי כסכום של מספרים מסדרת Fibonacci, למשל:

```
41=34+5+2, 7=5+2,  21=13+8, ...
```

כתבו פונקציה **רקורסיבית** **de_comp(n, base_lst)** אשר מקבלת מספר שלם אי-שלילי **n**, וכן **base_lst**, רשימה מקושרת שאיבריה הם סדרה סופית יורדת של איברי בסיס נומרי נתון (שלמים), בדומה לסדרת החזקות של 2 וסדרת איברי פיבונאצ'י (ראו בהמשך).

הפונקציה תחזיר רשימה מקושרת **חדשה** שאבריה מתוך ה-**base_lst** ,וסכומם הוא **n**. איברים אלה יופיעו ברשימה המוחזרת בסדר יורד, ללא חזרות, והאיבר הראשון ברשימה הוא המספר הגדול ביותר מבין אברי **base_lst** אשר קטן או שווה ל-**n**.

אם לא ניתן לקבל את הסכום של **n** מאברי **base_lst** בצורה הזו, הפונקציה תייצר חריגה (**Exception**) של **ArithmeticError**.

דוגמאות הרצה עם בסיס 2 ובסיס של סדרת פיבונאצ'י:

```
>>> twos = Link(32, Link(16, Link(8, Link(4, Link(2, Link(1))))))
>>> fibos = Link(34,(Link(21, Link(13, Link(8, Link(5, Link(3, Link(2,Link(1)))))))))

>>> de_comp(9, twos)
Link(8, Link(1))

>>> de_comp(45,twos)
Link(32, Link(8, Link(4, Link(1))))

>>> de_comp(71, twos)
ArithmeticError: ...

>>> de_comp(9, fibos) #Not 5+3+1
Link(8, Link(1))

>>> de_comp(30, fibos)
Link(21, Link(8, Link(1)))

>>> de_comp(41, fibos)
Link(34, Link(5, Link(2)))

>>> de_comp(0, fibos)
()
```

Answer:

(penalty regime: 0 %)

Reset answer

```
                                                                    :class Link
                                                                  () = empty

                                                          :(def __init__(self, data, next=empty
                                                                       self.data = data
                                                                       self.next = next

                                                                        :(def __repr__(self
                                                                 :if self.next is Link.empty
                                                       (return "Link({})".format(self.data
                                                                              :else
                                                 (return "Link({}, {})".format(self.data, self.next

                                               ######### !DO NOT EDIT ABOVE THIS LINE #########

                                                                    :(def  de_comp(n, base_lst
                                                                           ...
```

שאלה **9**

לא נענה

ניקוד השאלה: 15.00

סכום מבין רשימות

כתבו פונקציה **max_sum(liz1, liz2, liz3, lim)** אשר מקבלת שלוש רשימות לא ריקות (לאו דווקא ממוינות) של מספרים אי-שליליים, ומספר אי-שלילי **lim**. הפונקציה תחזיר את הסכום המקסימלי שקטן או שוה ל-**lim** אשר ניתן לקבל על ידי סכום של שלושה מספרים, כל אחד מרשימה אחרת מבין הרשימות **liz1**, **liz2**, **liz3**. אם אין אפשרות להחזיר מספר אשר קטן או שווה ל-**lim**, הפונקציה תחזיר 0.

דוגמאות:

```
>>> liz1 = [9, 8, 7]
>>> liz2 = [2, 5, 1]
>>> liz3 = [6]
>>> liz4 = [6, 2]
>>> liz5 = [0]

>>> max_sum(liz1, liz2, liz3, 30)
20

>>> max_sum(liz1, liz2, liz3, 20)
20

>>> max_sum(liz3, liz4, liz5, 13)
12

>>> max_sum(liz3, liz4, liz5, 7)
0
```

Answer:

(penalty regime: 0 %)

Reset answer

```
def max_sum(liz1, liz2, liz3, lim):
    ...
```

שאלה **10**

לא נענה

ניקוד השאלה: 15.00

מחלקת שברים

כתבו את המחלקה **Frac** באמצעותה ניתן לייצר אובייקטים אשר מייצגים שברים. שיטת האתחול של המחלקה כבר נכתבה לכם, ואין לשנות אותה.

```
class Frac:

    def __init__(self, mone, mahane):
        self.mone = mone

        if mahane == 0:
            raise ZeroDivisionError

        self.mahane = mahane
        self._reduce()
```

ניתן להשתמש בפונקציה **gcd**, ובפונקציה **lcm** שנספק לכם.

- שימו לב: בכל השיטות מלבד **_reduce** אין לשנות את האובייקט **self**.
- בהמשך, ארגומנטים בשם **other** הם אובייקטים מטיפוס **Frac** (וגם אותם אין לשנות).

ממשו את השיטות הבאות:

0:00

`_reduce(self)`:

תצמצם את הייצוג הפנימי של השבר אותו האובייקט מייצג. לא מחזירה דבר.

0:00

`__add__(self, other)`:

מחזירה אובייקט חדש מסוג **Frac** אשר מייצג את תוצאת החיבור של שבר אחר לשבר הנוכחי.

0:00 / 0:00

`__sub__(self, other)`:

מחזירה אובייקט חדש מסוג **Frac** אשר מייצג את תוצאת החיסור של שבר אחר מהשבר הנוכחי.

0:00 / 0:00

`__mul__(self, other)`:

מחזירה אובייקט חדש מסוג **Frac** אשר מייצג את תוצאת המכפלה של השבר הנוכחי עם שבר אחר.

0:00 / 0:00

`__truediv__(self, other)`:

מחזירה אובייקט חדש מסוג **Frac** אשר מייצג את תוצאת החילוק של השבר הנוכחי בשבר אחר.
במקרה של חלוקה באפס, יש לזרוק **ZeroDivisionError**.

0:00 / 0:00

`__pow__(self, power)`:

מחזירה אובייקט חדש מסוג **Frac** אשר מייצג את תוצאת העלאת השבר הנוכחי בחזקת המעריך.
ניתן להניח שהמעריך הוא מספר שלם.
שימו לב ש **x^-n=1/x^n** וגם **1=0^0**.
אין להשתמש באופרטור ** של פייתון, ולא בפונקציות **pow**.
במקרה של חלוקה באפס, יש לזרוק **ZeroDivisionError**.

0:00 / 0:00

`__eq__(self, obj)`:

מחזירה **True** אם **obj** הוא שבר מסוג **Frac** וערכו שוה לערך השבר הנוכחי. אחרת מחזירה **False**.
שימו לב שבשיטה זו לא ניתן להניח שהאוביקט השני הוא שבר.

0:00 / 0:00

`__ge__(self, other)`:

מחזירה **True** אם ערך השבר הנוכחי גדול או שוה לערך השבר האחר. אחרת מחזירה **False**

0:00 / 0:00

`__str__(self)`:

מחזירה מחרוזת אשר מציגה את השבר לפי הפורמט שבדוגמאות.

דוגמאות להדפסות:

```
>>> fr1 = Frac(21, 36)
>>> fr2 = Frac(13, 5)
>>> print(fr1)
7/12
>>> print(fr2)
2 3/5
>>> print(Frac(-3, 6))
-1/2
>>> print(Frac(3, -6))
-1/2
>>> print(Frac(-3, 2))
-1 1/2
>>> print(Frac(6, 3))
2 0/1
>>> print(Frac(0, 3))
0/1
```

0:00 / 0:00

דוגמאות לפונקציות האחרות:

```
>>> print(fr1+fr2)
3 11/60

>>> print(fr1-fr2)
-2 1/60

>>> print(fr2-fr1)
2 1/60

>>> print(fr1*fr2)
1 31/60

>>> print(fr2/fr1)
4 16/35

>>> print(fr1**2)
49/144

>>> print(fr2**-1)
5/13

>>> print(Frac(3, 6)==Frac(2, 4))
True

>>> print(fr1==fr2)
False

>>> print(fr1>=fr2)
False

>>> print(fr2>=fr1)
True
```

Answer:

(penalty regime: 0 %)

Reset answer

```
                                                                from math import gcd
        def lcm(a, b): return abs(a // gcd(a, b) * b) #intentionally fails when gcd(a,b)==0

                                                                                    :class Frac

                                                        :(def __init__(self, mone, mahane
                                                                    self.mone = mone

                                                                     :if mahane == 0
                                                          raise ZeroDivisionError

                                                                 self.mahane = mahane
                                                                       ()self._reduce
                                              ######### !DO NOT EDIT ABOVE THIS LINE #########

                                                                    :(def _reduce(self
                                                                                ...
```

שאלה **11**

לא נענה

לא ניתן ציון

תוכלו לרשום כאן הערות וטקסט חופשי במידת הצורך. **חלק זה אינו נבדק מראש** אלא רק במקרים חריגים ובמקרה של ערעורים.

שאלה **12**

לא נענה

ניקוד השאלה: 1.00

סמנו איזה שאלות בחרתם בחלק ב', השאלות שתסמנו הן אלו שיבדקו **בלבד**.

- שאלה 5 - עץ מונוטוני
- שאלה 6 - Equal Brackets
- שאלה 7 - עץ ביטוי
- שאלה 8 - de_comp
- שאלה 9 - סכום מבין רשימות
- שאלה 10 - Frac Class